

Scenario 5 — Premium Lapsed, Claim Blocked Before It Even Reaches the Oracle

Full mechanics: see Step-2 and Step-3. Steps 4 and 5 never happen — the claim is rejected on-chain immediately, before any off-chain oracle call is ever made.

Values used

	Value
Policyholder	Alice, 0x1111...aaaa
templateId	3 (Premium)
Premium paid	33 ether, once, at subscription
premiumPaidUntil[1]	$1_800_000_000 + 2_592_000 = 1_802_592_000$ (30 days)
premiumGracePeriodSeconds	604_800 (7 days)
Time of claim attempt	$1_803_500_000$ — after <code>premiumPaidUntil</code> + grace
Claim amount attempted	221 ether
Result	Reverts — never reaches <code>claims</code> storage at all

Walkthrough (deltas)

- **Step 2:** identical to Scenario 3 — Alice subscribes and pays once. Critically, she never calls `payPremium` again — no renewal.
- Time passes. $\text{premiumPaidUntil}[1] + \text{premiumGracePeriodSeconds} = 1_802_592_000 + 604_800 = 1_803_196_800$. At `block.timestamp = 1\_803\_500\_000`, this is already past that boundary.
- **Step 3, `submitClaim`**, walking through the same checks as the main trace:
 1. $221 \text{ ether} \leq \text{maxPayoutPerClaim}$ — passes.
 2. `getPolicy(1)` — returns the same `Policy` struct as before, still `revoked: false` and still within `[validFrom, validUntil]` (the policy's 365-day term hasn't expired — only the premium has lapsed, a distinct concept).
 3. `isPremiumCurrent(1)` — this is where it diverges.
`InsurancePolicy.isPremiumCurrent: premiumTerms[1].amountPerPeriod != 0` (billing is active, not exempt), so it evaluates $\text{block.timestamp } (1_803_500_000) \leq \text{premiumPaidUntil}[1] + \text{grace } (1_803_196_800) \rightarrow \text{false}$.
 4. Back in `ClaimRegistry.submitClaim: if (!isPremiumCurrent) revert PremiumNotCurrent();` — the transaction reverts here. Nothing after this line executes: no `Claim` struct is written, no `claimId` is assigned, `nextClaimId` doesn't advance, no event is emitted.

End state

```
claims[1]:      does not exist - the whole transaction reverted
nextClaimId:    unchanged
Alice's coverage: still nominally active (policy not revoked, not expired) but
unusable until she pays again
```

What fixes this

Alice calls `InsurancePolicy.payPremium(1)` again (restricted to the policyholder via

`OnlyPolicyHolderCanPay`). Per Step-2 's `_collectPremium` logic, since she's now paying after the old `premiumPaidUntil` has passed, the new payment extends from `block.timestamp` (now), not from the old lapsed date — she doesn't get retroactive coverage for the gap, only coverage going forward from whenever she actually pays.